Web Scraper for RAG Pipeline

Project Repository:
https://github.com/GautamKumar2005/Small-backend/tree/main/RAG

Overview

This project implements a highly resilient, professional-grade web scraper in Python. It is designed to systematically fetch data from practice sites to build a structured corpus suitable for a Retrieval-Augmented Generation (RAG) pipeline.

Key Features & Professional Standards

- **Robots.txt Compliance:** The scraper integrates Python's *urllib.robotparser* to automatically read and respect the target site's robots.txt directives, ensuring that crawling is strictly authorized.
- **Rate Limiting:** Built-in delays using *time.sleep()* ensure the scraper behaves politely, preventing the target server from being overwhelmed.
- **Network Resilience:** The requests session is wrapped with a custom *urllib3 Retry adapter*. If the server experiences a 500, 502, 503, or 504 error, the scraper will gracefully back off and retry instead of abruptly crashing.
- **Clean Data Extraction:** Uses *BeautifulSoup* to parse HTML, traverse pagination dynamically, and strip away styling and unicode quote characters. This guarantees that the downstream embedding models receive pristine text.
- **JSONL Format:** Extracted records (including text, author, tags, source URLs, and UTC scraped_at timestamps) are saved directly into a line-delimited JSON (JSONL) file. This is the industry standard format for efficiently streaming large RAG datasets.

Usage & Next Steps

Reviewers can view the code directly via the GitHub link provided above. The **corpus.jsonl** file within the repository contains the cleanly extracted sample data ready to be tokenized and ingested.